H. Paging & Address Translation (OSNo08 ท้าย) 🔥🔥

> **📖 รู้จักส่วนนี้ก่อนอ่าน — รีวิวรุ่นพี่: "คนส่วนมากคำนวณ Paging กันไม่ทัน"**
>
> **ความหมาย:** Paging (การแบ่งหน่วยความจำเป็นหน้าเท่า ๆ กัน) = หั่น RAM เป็น frame หั่นโปรแกรมเป็น page (ขนาดเท่ากัน) แล้วใช้ Page Table (ตารางจับคู่เพจ↔เฟรม) จับคู่ — "คำนวณ Paging" คือการแปลงที่อยู่ที่โปรแกรมเห็น (Logical) ให้เป็นที่อยู่จริงใน RAM (Physical)
>
> **ความสำคัญกับข้อสอบ:** แปลง address ทีละบิต, TLB (แคชตารางเพจความเร็วสูง) + สูตร EAT (เวลาเข้าถึงเฉลี่ย), Two-level paging — เป็นแนวคำนวณที่ตัดสินว่า "ทันสอบหรือไม่ทัน" ต้องฝึกจนข้อละไม่เกิน 1 นาที
>
> **ใช้จริงในคอมพิวเตอร์:** CPU ทุกตัวสมัยใหม่ (x86, ARM ในมือถือ) + OS ทุกตัว (Windows/Linux/macOS/Android) ใช้ paging จริง 100% — แอปแต่ละตัวเห็น "RAM ของตัวเอง" ก็เพราะกลไกนี้

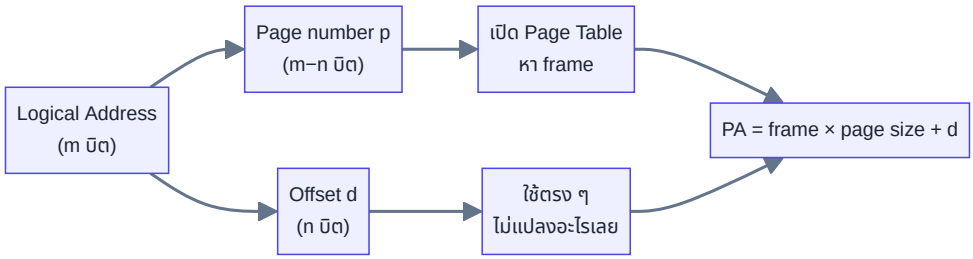
H1. แนวคิด Paging

- **Paging = จัดสรรแบบ non-contiguous** (ไม่ต้องติดกัน) — ตัด external fragmentation ขาด
- **Frame (เฟรม (ก้อน RAM))**: แบ่ง RAM เป็นบล็อกขนาดคงที่ ขนาดเป็นกำลังสอง 2^n bytes (512–8,192)
- **Page (หน้า)**: แบ่งโปรแกรมเป็นบล็อก**ขนาดเท่า frame เป๊ะ** · รันโปรแกรม n เพจ = หา frame ว่าง n อันกระจายวาง
- **Page table**: จับคู่ "เพจ i → frame ไหน" ของแต่ละ process

> **🎯 เช็คความเข้าใจ**
>
> Paging **ไม่มี External frag** แต่มี **Internal frag** — เกิดเฉพาะ**เพจสุดท้าย**ถ้าขนาดโปรแกรมไม่ลงตัว

H2. โครงสร้าง Logical Address — บิต p กับ d 🔥

Address ที่ CPU ปล่อย (ยาว **m บิต**) ซอยเป็น 2 ส่วนโดยฮาร์ดแวร์:



`Page size = 2ⁿ bytes → บิต offset = n บิต · บิต page number (หมายเลขเพจ) = m − n บิต → เพจได้สูงสุด 2^(m−n) เพจ`

ตัวอย่างวิดีโอ: m = 5 บิต (2^5 = 32 bytes), page = 4 bytes (2^2) → offset 2 บิต, page# 3 บิต → 8 เพจ (0–7 นับจากศูนย์!)

> **🪑 อุปมาอาจารย์**
>
> เก้าอี้ 32 ตัว จัด 8 ห้อง ห้องละ 4 — อ้างอิงว่า "เก้าอี้ตัวที่ 3 ในห้องเบอร์ 1" = (Page 1, Offset 3) สองเลขนี้คือทั้งชีวิตของ paging

H3. แปลง Logical → Physical ทีละขั้น 🔥

`Physical Address = (Frame Number × Page Size (ขนาดของเพจ)) + Offset`

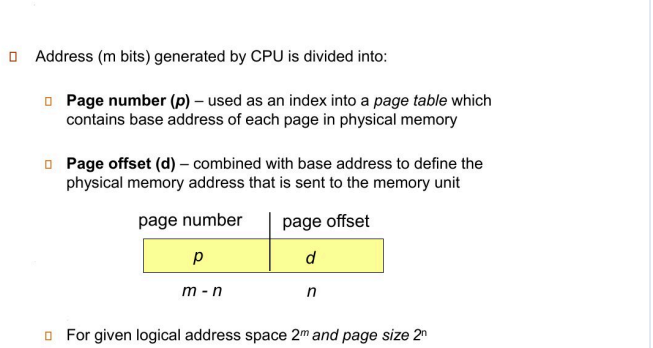
เปิดตารางด้วย p → ได้ frame → คูณขนาด frame = จุดเริ่ม → บวก offset

โจทย์จากสไลด์ p.32 (Paging Example): page size (ขนาดของเพจ) = 4 bytes · Page table: 0→5, 1→4, 2→1, 3→7

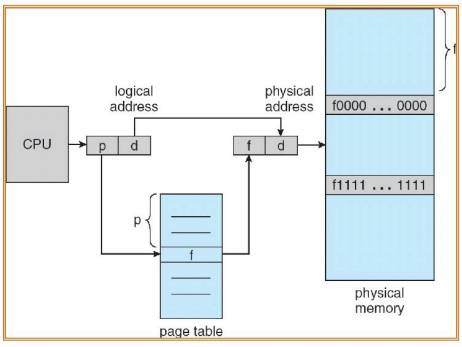
หาอะไร	Logical	ไล่ทีละขั้น	PA
ตัวอักษร 'C'	(p=0, d=2)	เพจ 0 → frame 5 → (5 × 4) + 2 = 20 + 2	**22**
ตัวอักษร 'K'	(p=2, d=2)	เพจ 2 → frame 1 → (1 × 4) + 2 = 4 + 2	**6**

🎯 3 จุดพลาดคลาสสิก

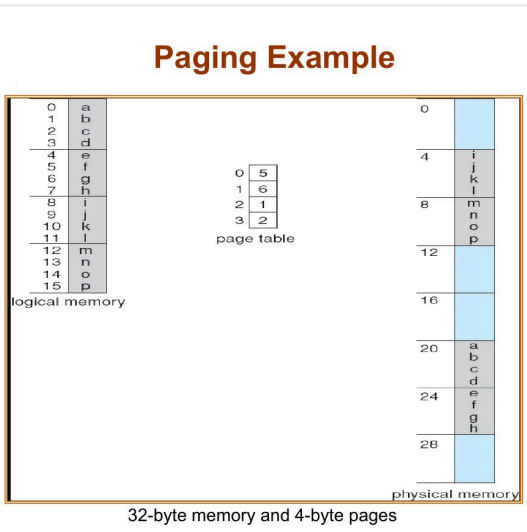
1) **บวก frame+offset ตรง ๆ** (5+2=7 ❌) ต้องคูณขนาดก่อน 2) นับเพจเริ่ม 1 → ต้องเริ่ม **Page 0** 3) ปัดจำนวนเพจลง — 42 bytes / เพจ 4 = 10.5 → **ปัดขึ้น 11 เพจ** (10 เพจเก็บได้ 40 bytes ข้อมูลขาด!) เพจสุดท้ายใช้ 2 bytes → internal frag = 4−2 = **2 bytes**



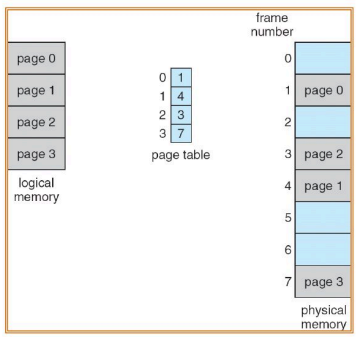
สไลด์ p.29 Address Translation Scheme (p | d)



สไลด์ p.30 Paging Hardware: p ไปตารางได้ frame + d ต่อกันเป็น PA



สไลด์ p.32 Paging Example — หน่วยความจำ 32 bytes, เพจ 4 bytes (โจทย์ 'C' และ 'K' อยู่นี่)



สไลด์ p.31 Paging Model: เพจกระจาย frame ไม่ติดกันก็ได้

H4. TLB + Effective Access Time 🔥

- Page table อยู่ใน RAM → อ่านข้อมูล 1 ครั้ง = ไป RAM **2 ครั้ง** (1: อ่านตารางหา frame, 2: อ่านข้อมูลจริง)
- **TLB** = แคชฮาร์ดแวร์ความเร็วสูง เก็บคู่ (page # → frame #) ล่าสุด ค้นแบบขนาน (associative memory)

กรณี	ไป RAM	เวลา (memory = 1 μs, TLB search = ε)
TLB Hit (โอกาส α)	1 ครั้ง	ε + 1 = 1 + ε
TLB Miss (โอกาส 1−α)	2 ครั้ง	ε + 1 + 1 = 2 + ε

$$EAT = \alpha(1 + \varepsilon) + (1 - \alpha)(2 + \varepsilon) = 2 + \varepsilon - \alpha$$

โจทย์วิดีโอ: ε = 0.1 µs, α = 80% → (0.8×1.1) + (0.2×2.1) = 0.88 + 0.42 = 1.3 µs (สูตรลัด: 2 + 0.1 − 0.8 = 1.3 ✓)

Associative Memory

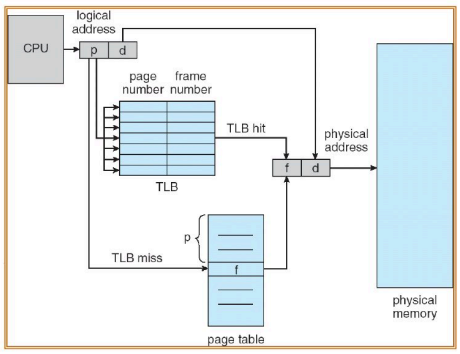
- Associative memory – parallel search

Page #	Frame #

Address translation (p, d)
- If p is in associative register, get frame # out
- Otherwise get frame # from page table in memory

สไลด์ p.35 Associative Memory (TLB): ค้นขนาน

Paging Hardware With TLB



สไลด์ p.36 Paging Hardware With TLB

H5. Protection — Valid / Invalid bit

- แต่ละแถว page table (ตารางจับคู่เพจ ↔ เฟรม) มี **valid–invalid bit**: **v** = เพจนี้อยู่ใน address space ของ process (ใช้ได้) · **i** = ห้ามอ้างอิง
- **PTBR** ชี้ตำแหน่ง page table ใน RAM · **PTLR** = ความยาวตาราง (เช็คเบื้องต้นว่าไม่อ้างเพจเกินขอบเขต)

🎯 จุดหลอก: valid ดู "ทั้งเพจ" ไม่ได้ดู "ทีละไบต์"

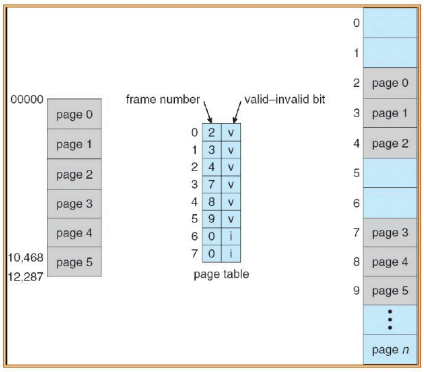
เพจสุดท้าย valid = v → ระบบยอมให้อ้าง offset *ใด ๆ* ในเพจนั้น (แม้เกินขนาดโปรแกรมจริง) — นี่คือเหตุผลที่ internal fragmentation ยังอยู่รอดใน paging

Memory Protection

- Memory protection implemented by associating protection bit with each frame
- **Valid-invalid** bit attached to each entry in the page table:
  - "valid" indicates that the associated page is in the process' logical address space, and is thus a legal page
  - "invalid" indicates that the page is not in the process' logical address space

สไลด์ p.38 Memory Protection

Valid (v) or Invalid (i) Bit In A Page Table

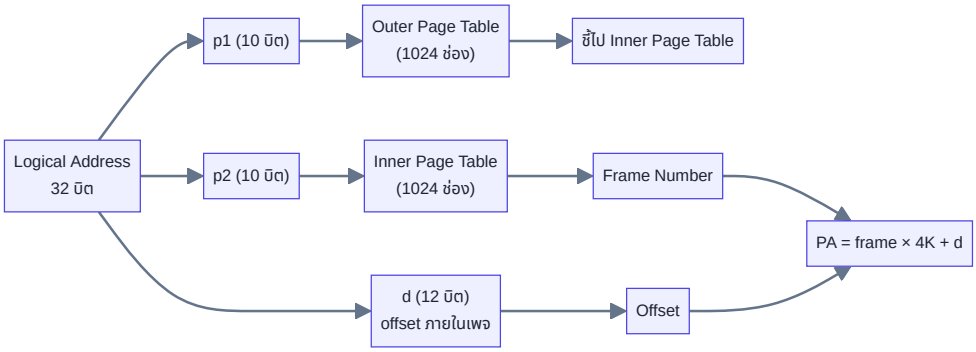


สไลด์ p.39 Valid (v) / Invalid (i) Bit

H6. Two-Level Paging — ทำไม + คำนวณ 🔥

ปัญหา single-level ในระบบ 32-bit: page 4KB (offset 12 บิต) → page# 20 บิต → 2^{20} = 1,048,576 ช่อง × 4B = **4MB ต่อ process ทุกตัว!** แม้โปรแกรมเล็กแค่ 1MB (ตารางต้องอยู่ต่อเนื่องด้วย)

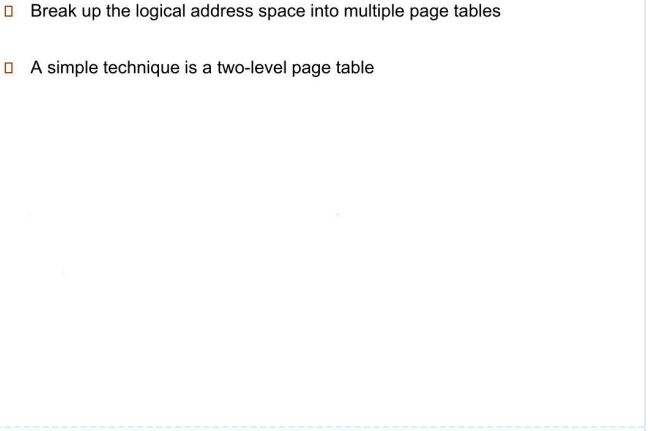
ทางแก้: ซอยบิต 32 ตัวเป็น **p_1 = 10 | p_2 = 10 | d = 12** — p_1 เปิด outer table หา inner table, p_2 เปิด inner table หา frame



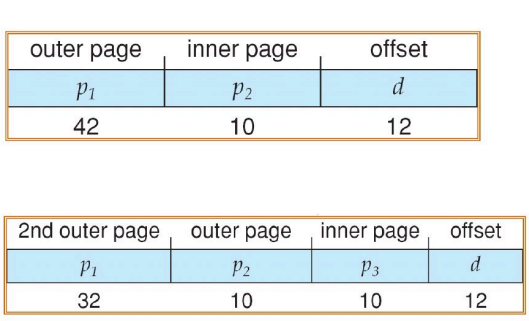
โจทย์ (จากวิดีโอ)	จำนวนเพจ	Inner tables ที่ต้องสร้าง	พื้นที่ page table รวม
โปรแกรม 1 MB	1024KB ÷ 4KB = **256 เพจ**	256 ≤ 1024 → **1 ตาราง**	outer 4KB + inner 4KB = **8 KB** (แทน 4MB!)
โปรแกรม 8 MB	8192KB ÷ 4KB = **2048 เพจ**	2048 ÷ 1024 = **2 ตาราง**	4KB + 2×4KB = **12 KB**

🟢 สูตรรวมทั้งบท

บิต offset = $\log_2$(page size) · บิต page# = บิต address − บิต offset · เพจ = ceil(ขนาดโปรแกรม ÷ page size) · PA = frame × pageSize + offset · EAT = 2 + ε − α · inner tables = ceil(จำนวนเพจ ÷ 1024)



สไลด์ p.42 Hierarchical Page Tables



สไลด์ p.44 Three-level Paging Scheme (x86 มีจริง)

Hashed Page Table — สำหรับ address space ใหญ่มาก

เมื่อ address space กว้างมาก (64-bit) ตารางเพจแบบลำดับชั้นจะลึกเกินไป จึงใช้ **ฟังก์ชัน hash** ของ page number ไปชี้ entry ในตาราง แล้วเดินตาม**ห่วงโซ่ (chain)** ของ element ที่ hash ชนกัน จนเจอแถวที่ page number ตรง แล้วอ่าน frame number — ดีกับเพจที่กระจายตัว ไม่ต้องสร้างตารางครบทุกช่อง

Inverted Page Table — 1 แถวต่อ 1 frame จริง

ปกติตารางเพจมี 1 แถวต่อ 1 *เพจเสมือน* ของ process (สร้างหลายตาราง) แต่ Inverted กลับด้าน: **1 แถวต่อ 1 frame จริงใน RAM** เก็บ (pid, page) ที่ครอบอยู่ — มีตารางเดียวสำหรับทั้งระบบ ประหยัด RAM มาก · ราคาที่จ่าย: ตอนแปลง address ต้อง**ค้นหา** (pid, page) ของตัวเองทั้งตาราง → ช้า จึงต้องมี hash table + TLB ช่วย

หมายเหตุ: ทั้งสองแบบเป็นแนวทางแก้ปัญหา "page table ใหญ่เกิน" ร่วมกับ Two-level ที่เจอข้างบน — ข้อสอบมักถามรู้จักชื่อ + หลักคิดสั้น ๆ แบบนี้พอ